

CHEAT SHEET

REACT + TYPESCRIPT



LAW'D
LIGA ACADÊMICA DE DESENVOLVIMENTO WEB



UNIVERSIDADE
FEDERAL DE
SERGIPE



Departamento de Computação/UFES

DECLARANDO UMA VARIÁVEL

TS



```
1  // constante
2  const nome: Tipo = valor;
3  // variável
4  let nome: Tipo = valor;
5  // variável
6  var nome: Tipo = valor;
```

DECLARANDO UMA FUNÇÃO



function



```
1 function nomeDaFuncao(arg1: Tipo1, arg2: Tipo2, ...): TipoRetorno {  
2     // execução da função  
3 }
```

arrow function



```
1 const nomeDaFuncao = (arg1: Tipo1, arg2: Tipo2, ...): TipoRetorno => {  
2     // execução da função  
3 }
```

DECLARANDO UMA FUNÇÃO

TS

arrow function



```
1 const nomeDaFuncao = (arg1: Tipo1, arg2: Tipo2, ...): TipoRetorno => valorRetorno
```


TIPOS PRIMITIVOS

TS



```
1  const nome: string = "Davi";  
2  const idade: number = 23;  
3  const ativo: boolean = true;
```

ARRAYS



```
1  const numeros: number[] = [1, 2, 3];  
2  // ou  
3  const nomes: Array<string> = ["Ana", "João"];
```

TUPLAS



```
1  const usuario: [string, number] = ["Maria", 30];
```

ENUMS

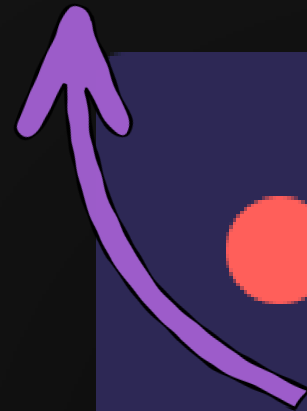


```
1  enum Status {  
2      PENDENTE,  
3      SUCESSO,  
4      ERRO,  
5  }
```

UNION TYPES



id pode ser número ou string



```
1  let id: number | string;  
2  id = 10;  
3  id = "10";
```


TYPE ALIAS

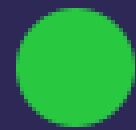
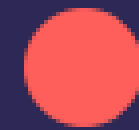
(TIPO PERSONALIZADO)



```
1  type User = {  
2    id: number;  
3    name: string;  
4  };
```

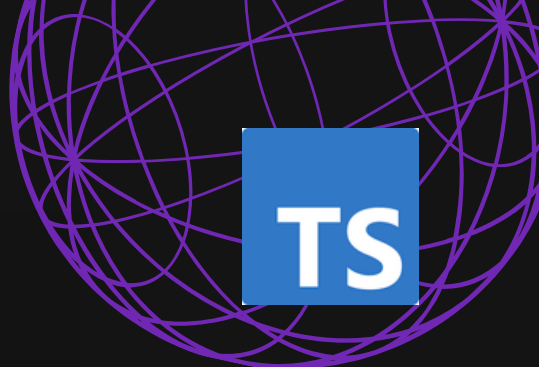
INTERFACES

TS



```
1 interface Product {  
2     id: number;  
3     price: number;  
4 }
```

OPTIONAL & READONLY

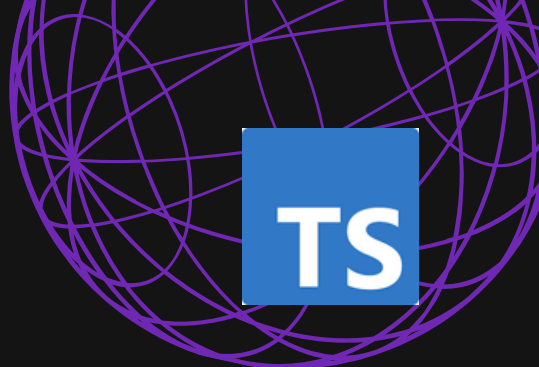


TS



```
1 interface User {  
2   id: number;  
3   // name pode existir ou não  
4   name?: string;  
5   // email não pode ser modificado  
6   readonly email: string;  
7 }
```


CLASSE

TS

```
1 class Usuario {
2   nome: string;
3   email: string;
4
5   constructor(nome: string, email: string) {
6     this.nome = nome;
7     this.email = email;
8   }
9
10  public saudacao(): string {
11    return `Olá, meu nome é ${this.nome}`;
12  }
13 }
14
15 const usuario = new Usuario("Davi", "davi@email.com");
16 console.log(usuario.saudacao());
```

MODIFICADORES DE ACESSO

TS

- **public**
- **private**
- **protected**

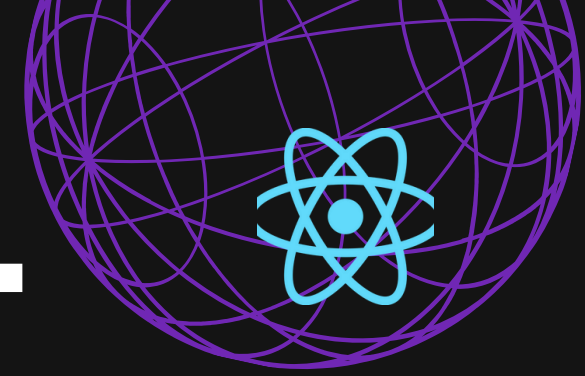
```
1  class Conta {
2      public dono: string;
3      private saldo: number;
4      protected numeroDaConta: string;
5
6      constructor(dono: string, saldoInicial: number) {
7          this.dono = dono;
8          this.saldo = saldoInicial;
9          this.numeroDaConta = "123-456";
10     }
11
12     protected formatarSaldo(): string {
13         return `R${this.saldo},00`;
14     }
15
16     public depositar(valor: number): void {
17         this.saldo += valor;
18     }
19
20     public getSaldo(): string {
21         return this.formatarSaldo();
22     }
23 }
```

HERANÇA

TS

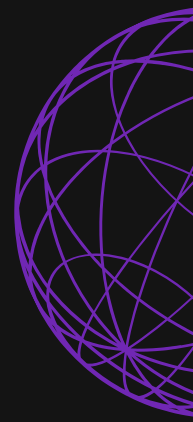
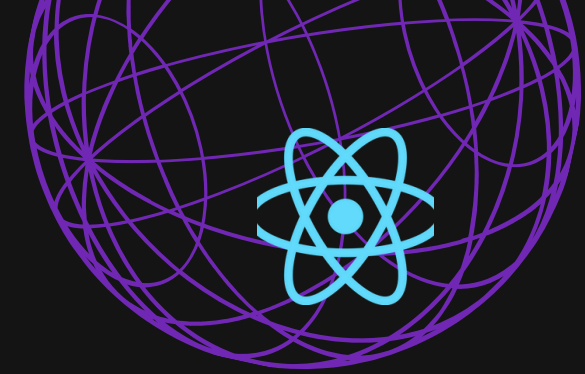
```
1 class ContaPoupanca extends Conta {
2     private taxaJuros: number;
3
4     constructor(dono: string, saldo: number, taxaJuros: number) {
5         super(dono, saldo);
6         this.taxaJuros = taxaJuros;
7     }
8
9     public aplicar(): void {
10         // acesso permitido ao protected
11         console.log(`Conta: ${this.numeroDaConta}`);
12     }
13 }
```

COMPONENTE FUNCIONAL



```
1 type Props = {
2   title: string;
3 };
4
5 // ou
6
7 interface Props {
8   title: string;
9 }
10
11 function Header({ title }: Props) {
12   return <h1>{title}</h1>;
13 }
14
15 // ou
16
17 const Header = ({ title }: Props) => {
18   return <h1>{title}</h1>;
19 };
20
21 const MeuComponente = () => {
22   return <Header title="Título" />;
23   // ou
24   return <Header title="Título"></Header>;
25 };
```

EVENTOS



```
1 function handleNomeDoEvento(event: MouseEvent<HTMLButtonElement>) {  
2     console.log("Execução do evento: ", event);  
3 }  
4  
5 // ...  
6 <button onClick={handleNomeDoEvento}>Meu botão</button>;
```


useState



função de atualização do estado



```
1 const [count, setCount] = useState<number>(0);
```

valor
armazenado

valor inicial

useEffect



```
1  useEffect(() => {  
2    console.log("Executou");  
3  }, []);
```

código a ser
executado

array de dependências
(o que dispara uma nova renderização)

useContext



definição
(criação)

```
1 const ThemeContext = createContext<string>("light");
```

```
1 const ComponentePai = () => {  
2   return <ThemeContext.Provider value="dark"></ThemeContext.Provider>;  
3 };
```

instanciação
através do
Provider

```
1 const ComponenteFilho = () => {  
2   const theme = useContext(ThemeContext);  
3   // theme = "dark";  
4   // ...  
5 };
```

utilização

PROGRAMAÇÃO FUNCIONAL

TS

map

método do objeto Array

assinatura do método



```
1 Array.map((valorAtual, indice, array) => {  
2     // processamento  
3     return novoValor;  
4 });
```

opcional

opcional

PROQAMAÇÃO FUNCIONAL

TS

map



```
1 const numeros: number[] = [1, 2, 3];  
2 const dobro: number[] = numeros.map((n) => n * 2);  
3 // [2, 4, 6]
```

executa um processamento
para cada item do array e
retorna um novo array com
a mesma quantidade de
itens.

função que retorna o dobro
do valor do array inicial

PROGRAMAÇÃO FUNCIONAL

TS

map

equivalente em programação
imperativa

```
1  const numeros: number[] = [1, 2, 3];  
2  const dobro: number[] = [null, null, null];  
3  for (let i = 0; i < numeros.length; i++) {  
4      dobro[i] = numeros[i] * 2;  
5  }
```

PROQAMAÇÃO FUNCIONAL

TS

filter

método do objeto Array

assinatura do método



```
1 Array.filter((valorAtual, indice, array) => {  
2     // processamento  
3     return true;  
4     // ou  
5     return false;  
6 });
```

opcional

opcional

PROGRAMAÇÃO FUNCIONAL

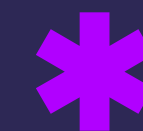
TS

filter



```
1 type Usuario = {
2   nome: string;
3   ativo: boolean;
4 };
5
6 const usuarios: Usuario[] = [
7   { nome: "Ana", ativo: true },
8   { nome: "João", ativo: false },
9   { nome: "Miguel", ativo: true },
10 ];
11
12 const usuariosAtivos: Usuario[] = usuarios.filter((u) => u.ativo);
13 /* [
14   *   { nome: "Ana", ativo: true },
15   *   { nome: "Miguel", ativo: true },
16   * ]
17 */
```

retorna um novo
array filtrado



a função deve
retornar um valor
booleano (**true** ou
false)

função que filtra
apenas usuários
ativos



LAWD
LIGA ACADÊMICA DE DESENVOLVIMENTO WEB



Departamento de Computação/UFES

PROGRAMAÇÃO FUNCIONAL

TS

filter

equivalente em programação
imperativa



```
1  const usuariosAtivos: Usuarios[] = [];  
2  
3  for (let i = 0; i < usuarios.length; i++) {  
4      if (usuarios[i].ativo) usuariosAtivos.push(usuarios[i]);  
5  }
```

PROGRAMAÇÃO FUNCIONAL

TS

reduce



```
1 Array.reduce((valorAnterior, valorAtual, indice, array) => {  
2   // processamento  
3   return novoValor;  
4 }, valorInicial);
```

opcional

opcional

PROGRAMAÇÃO FUNCIONAL

TS

reduce

processa os elementos do array e retorna um valor de qualquer outro tipo (number, string, TipoPersonalizado, Array, etc.)



```
1 const valores: number[] = [10, 20, 30];  
2 const sum: number = valores.reduce((acc, atual) => acc + atual, 0);
```

função que realiza a soma de todos os items

PROGRAMAÇÃO FUNCIONAL

TS

reduce

equivalente em programação
imperativa



```
1 let soma: number = 0;
2 const valores: number[] = [10, 20, 30];
3 for (let i = 0; i < valores.length; i++) {
4     soma += valores[i];
5 }
```

PROGRAMAÇÃO FUNCIONAL

TS

map + filter + reduce

```
1 type Produto = {
2   nome: string;
3   preco: number;
4   disponivel: boolean;
5 };
6
7 const produtos: Produto[] = [
8   { nome: "Mouse", preco: 100, disponivel: true },
9   { nome: "Teclado", preco: 200, disponivel: false },
10  { nome: "Monitor", preco: 900, disponivel: true },
11 ];
12
13 const total = produtos
14   .filter((p) => p.disponivel)
15   .map((p) => p.preco)
16   .reduce((acc, preco) => acc + preco, 0);
```

LINKS ÚTEIS

Web Dev Geral (W3Schools) - <https://www.w3schools.com/>

Web Dev Geral (MDN) - <https://developer.mozilla.org/pt-BR/>

Tutorial HTML - <https://www.w3schools.com/html/default.asp>

Referência Tags HTML - <https://www.w3schools.com/TAQS/default.asp>

Documentação TypeScript - <https://www.typescriptlang.org/docs/handbook/intro.html>

TypeScript Playground - <https://www.typescriptlang.org/pt/play>

Documentação React - <https://react.dev/learn>

React básico (useState) - <https://www.youtube.com/watch?v=mY9MLdifqe0>

React básico (useEffect) - https://www.youtube.com/watch?v=L6raE_UfwY&pp=ygUQcm9ja2V0c2VhdCBuZWJjdA%3D%3D

Curso React - <https://www.youtube.com/watch?v=FXpX7oof0I4&list=PLnDvRpP8BneyVA0SZ2okm-QBojomniQVO&index=1>

CANAIS YOUTUBE

Matheus Battisti - Hora de Codar - <https://www.youtube.com/@MatheusBattisti>

Rocketseat - <https://www.youtube.com/@rocketseat>